Software Engineering

LECTURE 4

DR.DINA FAWZY

Requirements Engineering

Requirement: A function, constraint or other property that the system must provide to fill the needs of the system's intended user(s).

Engineering: implies that systematic and repeatable techniques should be used.

Requirement Engineering means that requirements for a product are defined, managed and tested systematically.

- The process of finding out, analyzing, documenting and checking system requirements and constraints

Requirements Classifications

Types of requirement

User requirements (General):

- Statements, in a **natural language plus diagrams**, of what **services** the system is expected to provide to system users and the **constraints** under which it must operate.

System requirements (Specific):

- A **structured document** setting out detailed descriptions of the system's functions, services and operational constraints. Defines what should be implemented so may be **part of a contract** between client and contractor.

It shows how a user requirement may be expanded into several system requirements:

User Requirement Definition

1. The MHC-PMS shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

System Requirements Specification

1.1 On the last working day of each month, a summary of the drugs prescribed, their cost, and the prescribing clinics shall be generated.

1.2 The system shall automatically generate the report for printing after 17.30 on the last working day of the month.

1.3 A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed, and the total cost of the prescribed drugs.

1.4 If drugs are available in different dose units (e.g., 10 mg, 20 mg) separate reports shall be created for each dose unit.

1.5 Access to all cost reports shall be restricted to authorized users listed on a management access control list.

Functional and Non-Functional Requirements

Functional requirements

- Statements of **services** the system should provide, **how the system should react to particular inputs** and **how the system should behave in particular situations.**
- May state **what the system should not do**.

Non-functional requirements

- **Constraints** on the services or functions offered by the system such as **timing** constraints, constraints on the **development pr**ocess, standards, etc.
- Often apply to the **system** as a **whole** rather than individual features or services.

Functional requirements

Describe functionality or system services.

Depend on the **type** of software, expected **users** and the type of **system** where the software is used.

Functional **user** requirements may be **high-level** statements of **what** the system should do.

Functional **system** requirements should describe the system services **in detail**.

Non-Functional Requirements

These define system properties and **constraints** e.g. reliability, response time and storage requirements. Constraints are I/O device capability, system representations, etc.

Non-functional requirements may be more critical than functional requirements. If these are not met, the system may be **useless**.

Non-Functional Classifications

Product Requirements

- These requirements specify or constrain **the behavior of the software**.
- Examples include **performance** requirements on **how fast the system** must execute and how much **memory** it requires, reliability requirements that set out the **acceptable failure rate**, **security** requirements, and **usability** requirements.

Non-Functional Classifications

Organizational requirements

- These requirements are broad system requirements derived from **policies and procedures in the customer's and developer's organization**.
- Examples include operational process requirements that define how the system will be used, development process requirements that specify the programming language, the development environment or process standards to be used, and environmental requirements that specify the operating environment of the system.

Non-Functional Classifications

External Requirements

- Requirements which arise from factors which are **external to the system and its development process.**
- These may include **regulatory requirements** that set out what must be done for the system to be approved for use by a regulator, such as a central bank; legislative requirements that must be followed to ensure that the system **operates within the law; and ethical requirements** that ensure that the system will be acceptable to its users and the general public.

Examples of Non-Functional Requirements in the MHC-PMS

Product requirement
The MHC-PMS shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.

Organizational requirement
Users of the MHC-PMS system shall authenticate themselves using their health authority identity card.

External requirement
The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

Requirements in agile

The **starting point for planning each increment** is establishing **the product backlog**, which is the **list of work to be done on the project**.

- During **this**, requirements are determined and the **customer is closely involved in this process** and can introduce **new requirements or tasks at the beginning of each increment**.

Goals and Requirements

Non-functional requirements may be very difficult to state precisely and imprecise requirements may be **difficult to verify**.

Goal

- A **general intentio**n of the user such as ease of use, the ability of the system to recover from failure, or rapid user response.

Verifiable non-functional requirement

- A statement using some **measure** that can be objectively **tested**.

Goals are helpful to developers as they convey the **intentions** of the system users.

Example: Usability Requirements

Goal

- The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized.

Testable non-functional requirement

- Medical staff shall be able to use all the system functions after four hours of training.
- After this training, the average number of errors made by experienced users shall not exceed two per hour of system use.

Metrics for Specifying Non-Functional Requirements

Property	Measure
Speed	Processed transactions/second User/event response time Screen refresh time
Size	Mbytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Requirements Engineering Processes

Requirements Engineering Processes

Feasibility study An estimate is made of whether the identified user needs may be satisfied using current software and hardware technologies.

- The study considers whether the proposed system will be cost-effective from a business point of view and if it can be developed within existing budgetary constraints.
- A feasibility study should be relatively cheap and quick. The result should inform the decision of whether or not to go ahead with a more detailed analysis.

Requirements Elicitation and Analysis: Draw out the requirements from stakeholders.

- This is the process of deriving the system requirements through observation of existing systems, discussions with potential users and procurers, task analysis, and so on. This may involve the development of one or more system models and prototypes. These help you understand the system to be specified.

Requirements Engineering Processes

Requirements specification

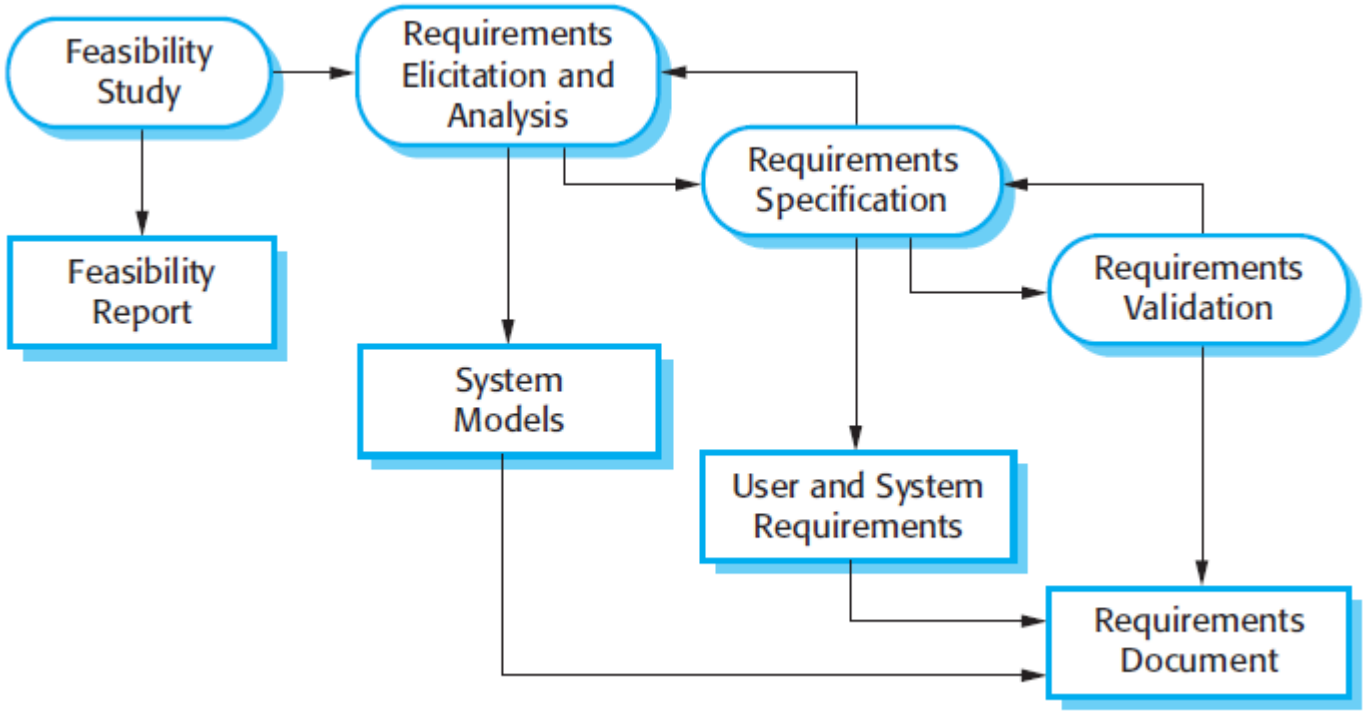
- Requirements are **documented**.

Requirements Validation: Review the requirement specification for errors, ambiguities, omissions, and conflicts.

Requirements Management: Manage changing requirements.

Requirements Engineering Processes

In practice, RE is an iterative activity in which these processes are interleaved.



Requirements Elicitation and Analysis

It involves software engineers working with a range of system stakeholders to find out about the application domain, the services that the system should provide and the system's operational constraints.

Identify stakeholders

- Stakeholder can be "**anyone who benefits in a direct or indirect** way from the system which is being developed"

Ex. Business manager, project manager, marketing people, software engineer, support engineer, end-users, internal-external customers, consultants, maintenance engineer.

- Each one of them has **different view of the system**.

Stakeholders in the MHC-PMS

Patients whose information is recorded in the system.

Doctors who are responsible for assessing and treating patients.

Nurses who coordinate the consultations with doctors and administer some treatments.

Medical receptionists who manage patients' appointments.

IT staff who are responsible for installing and maintaining the system.

Problems

Stakeholders are not completely **sure** of what is needed.

Stakeholders express requirements in their **own terms**.

Different stakeholders may have **conflicting** requirements.

Stakeholders have a **poor understanding** of the capabilities and limitations of the computing environment.

Organisational and political factors may influence the system requirements.

The requirements **change** during the analysis process.

Interviewing

Formal or informal interviews with stakeholders are part of most RE processes.

Types of interview

- **Closed interviews** based on pre-determined list of questions.
- **Open interviews** where various issues are explored with stakeholders.

Normally a **mix** of closed and open-ended interviewing.

Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.

Interviews are not good for understanding domain requirements

- Requirements engineers cannot understand specific domain terminology;

Scenarios

Scenarios are **real-life examples** of how a system can be used.

They should include

- A description of the **starting** situation;
- A description of the **normal** flow of events;
- A description of what can go **wrong**;
- Information about other **concurrent activities**;
- A description of the state when the scenario **finishes**.

Ethnography

A social scientist spends a considerable time observing and analysing how people actually work.

The value of ethnography is that it helps discover implicit system requirements (Social and organisational factors) that reflect the actual ways that people work, rather than the formal processes defined by the organization.

People often find it very difficult to articulate details of their work because it is second nature to them. They understand their own work but may not understand its relationship to other work in the organization.

Ethnography is effective for understanding existing processes but cannot identify new features that should be added to a system.

Requirement Specification

Requirement Specification

Requirements specification is the process of **writing down** the **user and system requirements** in a requirements **document**.

Ideally, the user and system requirements **should be clear**, **unambiguous, easy to understand, complete, and consistent**.

In practice, this is difficult to achieve as stakeholders interpret the requirements in different ways and there are often inherent conflicts and inconsistencies in the requirements.

Software Requirements Specification (SRS)

The SRS is the **official statement** of what is required of the system developers.

It should include both a definition of **user requirements** and a specification of the **system requirements.**

It is **NOT** a design document. As far as possible, it should set of **WHAT** the system should do rather than HOW it should do it.

It contains a complete information description, a detailed functional description, a representation of system behavior, an indication of performance requirements and design constraints, appropriate validation criteria, and other information pertinent to requirements.

Requirement Validation

Requirements Validation

A requirements review is a process where a group of people from the system customer and the system developer read the requirements document in detail and check for errors, anomalies, and inconsistencies.

Once these have been detected and recorded, it is then up to the customer and the developer to negotiate how the identified problems should be solved.

The cost of fixing a requirements problem by making a system change is usually much greater than repairing design or coding errors.

Requirements checking

Validity. Does the system provide the functions which best support the customer's needs?

Consistency. Are there any requirements conflicts?

Completeness. Are all functions required by the customer included?

Realism. Can the requirements be implemented given available budget and technology?

Verifiability. Is the requirement realistically testable?

Comprehensibility. Is the requirement properly understood?

Traceability. Is the origin of the requirement clearly stated?

Requirements Validation Techniques

Requirements reviews

- Systematic **manual** analysis of the requirements.

Prototyping

- Using an **executable model** of the system to check if it meets their real requirements.

Test-case generation

- Developing **tests** for requirements to check testability.

Requirements Management

Requirements Management

Requirements management is the process of **understanding and controlling changes to requirements.**

New requirements emerge as a system is being developed and after it has gone into use.

The **business and technical environment of the system always changes after installation.**

- **New hardware** may be introduced, it may be necessary to interface the system with other systems, **business priorities** may change (with consequent changes in the system support required), and **new legislation and regulations** may be introduced that the system must necessarily abide by.

Requirements Change Management

Deciding if a requirements change should be accepted

- **Problem analysis and change specification**
  - During this stage, the problem or the change proposal is analyzed to check that it is **valid**. This analysis is fed back to the change requestor who may respond with a more specific requirements change proposal, or decide to withdraw the request.
- **Change analysis and costing**
  - The **effect** of the proposed change is assessed using traceability information and general knowledge of the system requirements. Once this analysis is completed, a decision is made whether or not to proceed with the requirements change.
- **Change implementation**
  - The requirements **document** and, where necessary, the system **design** and **implementation**, are **modified**. Ideally, the document should be organized so that changes can be easily implemented.

SRS Document Template

(Team ID) - Project Title

Supervisor TA
Group Members and sections
Date

1. Introduction

This section should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software.

2. Context Diagram

This section presents the context diagram for the system that identifies the flows of information between the system and external entities.

3. User Requirements

This section lists the requirements described by the user using his language.

4. Functional Requirements

This section lists the functional requirements in ranked order. Functional requirements describe the possible effects of a software system, in other words, what the system must accomplish. Each functional requirement should be specified in a format similar to the following: (this should be done for each functional requirement)

Functional requirement Name:

- Description/ Action
- Requirements/ Inputs
- Source (source of inputs that will be entered into the function)
- Pre-condition (what should be true so that function can start)
- Post-condition (what will happen after the function execution)
- Output (what will be displayed/generated after function execution)

A minimum number to be described is 10 different functional requirements.
Note: [Add, Edit, and Delete are not considered as functional requirements]

5. Non-Functional Requirements

Describe how the system accomplishes its functional requirements. Other kinds of requirements which specify any other particular non-functional attributes required by the system such as interface requirements, performance requirements, or reliability requirements.

Examples: Reliability, response time, execution speed, storage requirements, constraints, I/O device capability, system representations, mandating a particular IDE, programming language, and development method.

A minimum number to be described is 10 different non-functional requirements.

Thanks for Listening

ANY QUESTIONS!